



upGrad

Raho Ambitious

#LifeKoKaroLift



SGC Coaching Session 25:

Text Extracting and Cleaning Agenda
[Tutorial Session]

Setting the tone of the session

Learning objectives/takeaways of the session

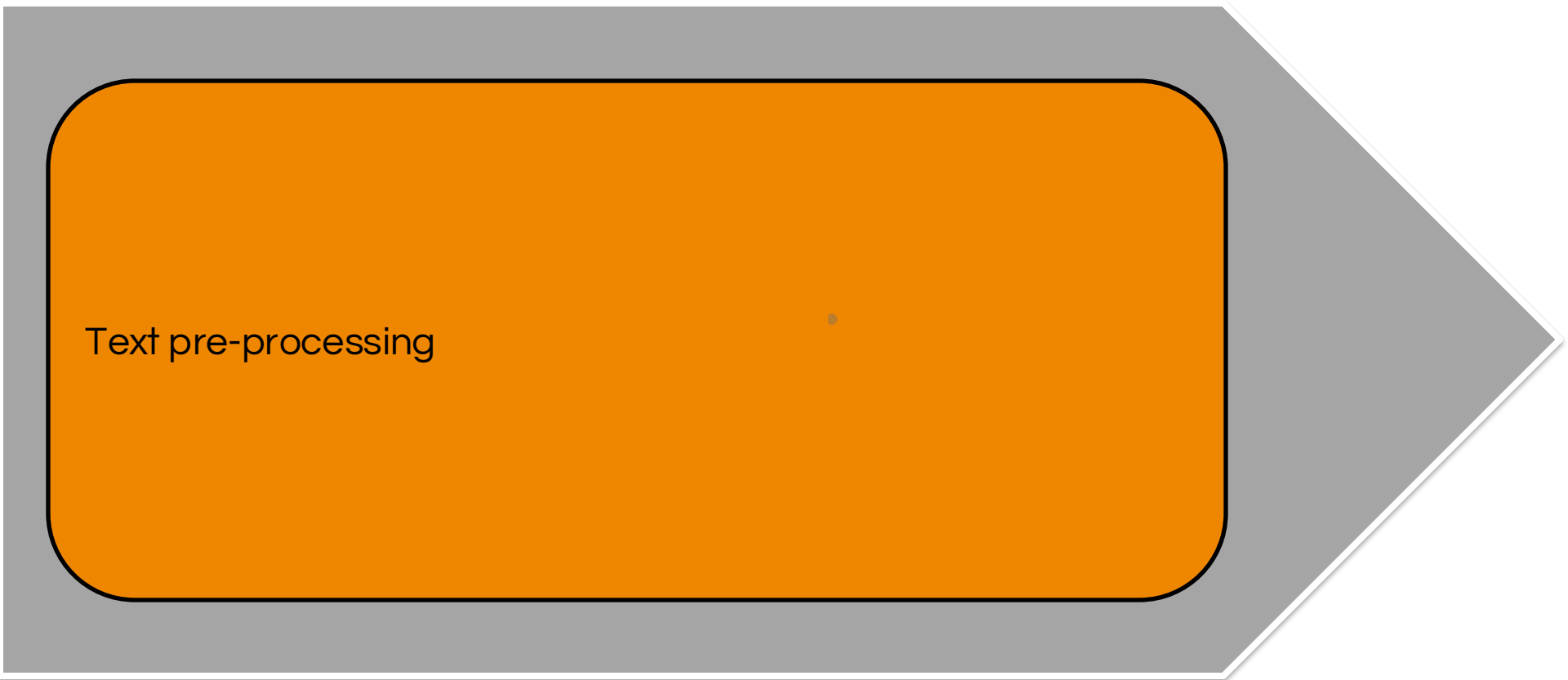
Focused teaching (via Live Coding,
demonstration)

Personalized Feedback and Doubt
Resolution + Conceptual
understanding of the approach

Pipelines of a NLP problem

Extracting Text and cleaning it

Doubt clearing



Text pre-processing

Noise Removal & Normalization

- **Special character cleaning:** special characters such as “\n” double quotes must be removed from the text since we aren’t expecting any predicting power from them.
- **Uppcase/downcase:** we would expect, for example, “Book” and “book” to be the same word and have the same predicting power. For that reason we have downcased every word.
- **Punctuation signs:** characters such as “?”, “!”, “;” have been removed.
- **Possessive pronouns:** in addition, we would expect that “Trump” and “Trump’s” had the same predicting power.
- **Stop words:** words such as “what” or “the” won’t have any predicting power since they will presumably be common to all the documents. For this reason, they may represent noise that can be eliminated. We have downloaded a list of English stop words from the *nltk* package and then deleted them from the corpus.

- Lemmatization is closely related to stemming.
- In linguistics, it is the process of grouping together the different inflected forms of a word so they can be analyzed as a single item.
- Lemmatization changes the verb form, while keeping the meaning of the word the same.
- Stemming will reduce “Worker” and “Speaker” to “Work” and “Speak” respectively while lemmatization will reduce them to “Worker” and “Speaker” because “Worker” is not an inflected form of “Work”, neither is “Speaker” an inflected form of “Speak”.

Original	Stemmed	Lemmatized
visibilities	visibl	visibility
adhere	adher	adhere
adhesion	adhes	adhesion
appendicitis	append	appendicitis
oxen	oxen	ox
indices	indic	index
swum	swum	swim

- Stemming, in literal terms, is the process of cutting down the branches of a tree to its stem.
 - Language dependent
 - e.g., **automate(s), automatic, automation** all reduced to **automat**.
- Stemmer operates on a single word without knowledge of the context, and therefore cannot discriminate between words which have different meanings depending on part of speech.
- However, stemmers are typically easier to implement and run faster, and the reduced accuracy may not matter for some applications.
- Porter's Algorithm
 - rule based stemmer
 - produces stems, makes a number of mistakes
- Krovetz Algorithm
 - algorithm + dictionary
 - if not found → strip suffix, lookup again
 - produces "words" not stems. light in nature and can be used as pre-stemmer for other stemmers.
 - It is inefficient in case of large documents.

- TF-IDF stands for “Term Frequency — Inverse Document Frequency”
 - It gives us the frequency of the word in each document in the corpus.
 - It is the ratio of number of times the word appears in a document compared to the total number of words in that document.
 - It increases as the number of occurrences of that word within the document increases.
 - Each document has its own term frequency.

Example

Sentence 1 : The car is driven on the road.

Sentence 2 : The truck is driven on the highway.

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$

$tf_{i,j}$ = number of occurrences of i in j

df_i = number of documents containing i

N = total number of documents

Word	TF		IDF	TF*IDF	
	A	B		A	B
The	1/7	1/7	$\log(2/2) = 0$	0	0
Car	1/7	0	$\log(2/1) = 0.3$	0.043	0
Truck	0	1/7	$\log(2/1) = 0.3$	0	0.043
Is	1/7	1/7	$\log(2/2) = 0$	0	0
Driven	1/7	1/7	$\log(2/2) = 0$	0	0
On	1/7	1/7	$\log(2/2) = 0$	0	0
The	1/7	1/7	$\log(2/2) = 0$	0	0
Road	1/7	0	$\log(2/1) = 0.3$	0.043	0
Highway	0	1/7	$\log(2/1) = 0.3$	0	0.043

$$W_{t,d} = \begin{cases} 1 + \log_{10}(tf_{t,d}) & , \text{ if } tf_{t,d} > 0 \\ 0 & , \text{ Otherwise} \end{cases}$$

$$idf_t = \log_{10} (N/df_t)$$

$$W_{t,d} = [1 + \log_{10}(tf_{t,d})] \times [\log_{10} (N/df_t)]$$

Term Frequency for Ranked Retrieval

Query (q): Analytics book

Doc 1 (d1): This book is on
Analytics

Doc 2 (d2): Big-Data Analytics is
the process of examining large
volume of data

Term	Doc1	Doc2
Analytics	1	1
Big-Data	0	1
book	1	0
data	0	1
examining	0	1
is	1	1
large	0	1
of	0	2
on	1	0
process	0	1
the	0	1
this	1	0
volume	0	1

Term Frequency for Ranked Retrieval

Query (q): Analytics book

Doc 1 (d1): This book is on Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	Doc1	Doc2
Analytics	1	1
Big-Data	0	1
book	1	0
data	0	1
examining	0	1
is	1	1
large	0	1
of	0	1.3
on	1	0
process	0	1
the	0	1
this	1	0
volume	0	1

Term Frequency for Ranked Retrieval

Query (q): Analytics book

Doc 1 (d1): This book is on Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	Doc1	Doc2
Analytics	1	1
Big-Data	0	1
book	1	0
data	0	1
examining	0	1
is	1	1
large	0	1
of	0	1.3
on	1	0
process	0	1
the	0	1
this	1	0
volume	0	1

Score (d1) = 2

Score (d2) = Sum over terms in both query and doc 2



Term Frequency for Ranked Retrieval

Query (q): Analytics book

Doc 1 (d1): This book is on
Analytics

Doc 2 (d2): Big-Data Analytics is
the process of examining large
volume of data

Term	Doc1	Doc2
Analytics	1	1
Big-Data	0	1
book	1	0
data	0	1
examining	0	1
is	1	1
large	0	1
of	0	1.3
on	1	0
process	0	1
the	0	1
this	1	0
volume	0	1

Score (d1) = 2

Score (d2) = 1

Challenge: Term Frequency

Query (q): book of Analytics

Doc 1 (d1): This book is on Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	Doc1	Doc2
Analytics	1	1
Big-Data	0	1
book	1	0
data	0	1
examining	0	1
is	1	1
large	0	1
of	0	1.3
on	1	0
process	0	1
the	0	1
this	1	0
volume	0	1

Score (d1) = 2

Score (d2) = 2.3

Inverse Document Frequency (idf)

$$\text{idf}_t = \log_{10} (N/\text{df}_t)$$

N = Total number of documents

N = 1000

Term	df_t	idf_t
Analytics	100	1
book	10	$\log_{10}(100)$
is	800	
of	500	
the	1000	

Inverse Document Frequency (idf)

$$\text{idf}_t = \log_{10} (N/\text{df}_t)$$

N = Total number of documents

N = 1000

Term	df_t	idf_t
Analytics	100	1
book	10	2
is	800	0.09
of	500	0.3
the	1000	0

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	df(t)
Analytics	2
Big-Data	1
book	1
data	1
examining	1
is	2
large	1
of	2
process	1
the	1
this	1
volume	1

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	df(t)	idf(t)
Analytics	2	0
Big-Data	1	0.3
book	1	0.3
data	1	0.3
examining	1	0.3
is	2	0
large	1	0.3
of	2	0
process	1	0.3
the	1	0.3
this	1	0.3

Score (d1) = Sum over terms in both query and doc 1

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	df(t)	idf(t)
Analytics	2	0
Big-Data	1	0.3
book	1	0.3
data	1	0.3
examining	1	0.3
is	2	0
large	1	0.3
of	2	0
process	1	0.3
the	1	0.3

$$\text{Score (d1)} = 0 + 0.3 + 0$$

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	df(t)	idf(t)
Analytics	2	0
Big-Data	1	0.3
book	1	0.3
data	1	0.3
examining	1	0.3
is	2	0
large	1	0.3
of	2	0
process	1	0.3
the	1	0.3

Score (d1) = 0.3

Score (d2) = 0 + 0

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	d1(tf-idf)	d2(tf-idf)
Analytics	0	0
Big-Data	0	0.3
book	0.3	0
data	0	0.3
examining	0	0.3
is	0	0
large	0	0.3
of	0	0
process	0	0.3
the	0	0.3
this	0.3	0
volume	0	0.3

Query (q): book of Analytics

Doc 1 (d1): This book is of Analytics

Doc 2 (d2): Big-Data Analytics is the process of examining large volume of data

Term	d1(tf-idf)	d2(tf-idf)
Analytics	0	0
Big-Data	0	0.3
book	0.3	0
data	0	0.3
examining	0	0.3
is	0	0
large	0	0.3
of	0	0
process	0	0.3

Score (d1) = 0.3

Score (d2) = 0

Demonstration: Industry based examples for text cleaning and extraction

Questions/
Doubts?

